

---

## EVALUACIÓN FINAL

### Matemática Numérica • Curso 2025–2026

---

**Fecha de entrega:** 17 de junio de 2026

**Entrega:** Comprimido .zip

#### Instrucciones generales

1. Los ejercicios se resuelven en el Jupyter Notebook proporcionado (Evaluacion\_Final\_MatNum.ipynb). Complete las celdas de código donde se indica # su código aquí.
2. **Prohibido** usar `numpy.linalg.inv` para resolver sistemas lineales; use `numpy.linalg.solve` o sustitución directa.
3. Los ejercicios teóricos pueden desarrollarse con **lápiz y papel**. Adjunte el desarrollo (foto o escaneo) **dentro del mismo comprimido** (.zip) que entregue.
4. Se permite el uso de documentación oficial, apuntes del curso y recursos bibliográficos.
5. Los **Ejercicios Extra** no son necesarios para aprobar, pero son **imprescindibles** para obtener la calificación de 4 ó 5. Sin ellos no es posible alcanzar la nota máxima.

#### Descripción de los ejercicios

##### Ejercicio 1 — Aritmética de Punto Flotante (teórico-práctico)

Representación de  $-13,625$  en IEEE 754 de precisión simple (32 bits). Cálculo de errores absoluto y relativo de la aproximación de  $\pi$ . Análisis del fenómeno de *cancelación catastrófica* en  $f(x) = \sqrt{x+1} - \sqrt{x}$  con propuesta de versión numéricamente estable y comparación gráfica. *Las partes teóricas pueden desarrollarse con lápiz y papel.*

##### Ejercicio 2 — Sistemas de Ecuaciones Lineales (teórico-práctico)

**Parte a)** Dado  $Ax = b$  con factorización  $A = LU$  conocida, resolver sin calcular ninguna inversa matricial: sustituciones progresiva ( $Ly = b$ ) y regresiva ( $Ux = y$ ). Desarrollo analítico paso a paso e implementación en Python para verificar. *El desarrollo teórico puede hacerse con lápiz y papel.*

**Parte b)** Problema real de una fábrica de muebles: se debe identificar el sistema  $3 \times 3$  a partir del enunciado, construir la matriz de coeficientes y el vector de términos independientes, y resolver con `numpy.linalg.solve`.

**Ejercicio 3 — Ecuación No Lineal** (práctico)

Búsqueda de todas las raíces de  $f(x) = e^{-x/3}\cos(2x) - 0,3$  en  $[0, 10]$  usando `scipy.optimize.newton` con la derivada analítica. Análisis gráfico previo para estimar puntos iniciales. Verificación de que  $|f(\text{raíz})| < 10^{-9}$ .

**Ejercicio 4 — Interpolación** (práctico)

Interpolación de datos de concentración de  $\text{CO}_2$  (8 puntos, meses 0–12) mediante polinomio de Lagrange (`scipy.interpolate.lagrange`) y spline cúbico (`CubicSpline`). Comparación gráfica de ambos interpolantes y análisis del fenómeno de Runge.

**Ejercicio 5 — Regresión Lineal** (práctico)

Ajuste de datos de rendimiento académico (30 estudiantes) por mínimos cuadrados. Construcción manual de la matriz de diseño  $X$  y resolución de la ecuación normal  $(X^\top X)\hat{\beta} = X^\top y$  con `numpy.linalg.solve`. Comparación de modelos de 1 y 2 variables con visualización 3D del plano ajustado.

---

**Ejercicios Extra — Para la calificación máxima**

*Estos ejercicios no son necesarios para aprobar, pero son **imprescindibles** para alcanzar la calificación de 4 ó 5. Sin ellos no es posible obtener la nota máxima.*

**Extra 1 — Método de Newton desde cero**

Implementación del método de Newton–Raphson para  $f: \mathbb{R} \rightarrow \mathbb{R}$  sin usar `scipy`, únicamente con NumPy: criterio de parada por tolerancia, detección de derivada nula y registro del historial de convergencia. Verificación con la función del Ejercicio 3 y gráfico de convergencia cuadrática.

**Extra 2 — Regresión lineal  $n$  variables desde cero**

Implementación de `regresion_lineal(X, y)` con NumPy: construcción de  $X^\top X$  y  $X^\top y$ , resolución sin inversa. Verificación de que reproduce los resultados del Ejercicio 5.